

Task Overview

You are building a database for a **city metro system** that manages passengers, stations, and tickets.

Task 1: Create Database & Tables

Create Database

Create a database named:

metro_db

Create Tables

Table 1: stations

Column Name Data Type Constraint

station_id INT PRIMARY KEY, AUTO_INCREMENT

station_name VARCHAR(50) UNIQUE, NOT NULL

zone VARCHAR(10) NOT NULL

Table 2: passengers

Column Name Data Type Constraint

passenger_id INT PRIMARY KEY, AUTO_INCREMENT

name VARCHAR(50) NOT NULL

phone VARCHAR(10) UNIQUE

age INT CHECK (age >= 5)

gender VARCHAR(10)

Table 3: tickets

Column Name Data Type Constraint

ticket_id INT PRIMARY KEY, AUTO_INCREMENT

passenger_id INT FOREIGN KEY REFERENCES passengers(passenger_id) source_station

INT FOREIGN KEY REFERENCES stations(station_id) destination_station INT FOREIGN

KEY REFERENCES stations(station_id) fare INT CHECK (fare > 0)

travel_date DATE

Column Name Data Type Constraint

ticket_type VARCHAR(20) (Single / Return / Monthly)

Task 2: Insert Sample Data

1. Insert **at least 6 stations** with different zones
2. Insert **8 passengers** with different ages
3. Insert **10 ticket records** covering:
 - Different routes
 - Different ticket types
 - Different fares

Task 3: Basic Queries

Write queries to:

1. Display all stations
2. Display all passengers
3. Display all tickets
4. Show passenger name and fare paid
5. Display tickets booked on **today's date**

Task 4: Filtering & Conditions

1. Find passengers aged **above 60**
2. Find tickets with fare **greater than 50**
3. Display tickets of type **Monthly**
4. Find passengers who booked tickets from a specific station

Task 5: Sorting & Limiting

1. Display tickets sorted by fare (highest first)
2. Show **top 3 most expensive tickets**
3. Display passengers sorted by age (youngest to oldest)

Task 6: Aggregate Functions

Write queries to:

1. Find **total revenue** collected
2. Find **average ticket fare**
3. Find **maximum and minimum fare**
4. Count number of tickets per ticket type
5. Count passengers zone-wise (using joins)

Task 7: Join Operations (Important)

1. Display passenger name, source station, destination station, and fare
2. Display ticket details along with passenger phone number
3. Display tickets along with source and destination station names

Task 8: Update & Delete Operations

1. Increase fare by **10** for tickets below **30**
2. Update ticket type from Single to Return for a passenger
3. Delete tickets booked before a specific date

Task 9: Subqueries (Interview Focus)

1. Find passengers who paid **more than average fare**
2. Find station(s) from where **maximum tickets** were booked
3. Find passenger(s) who booked **more than 1 ticket**

Task 10: View Creation

1. Create a view **daily_revenue** showing:
 - travel_date
 - total fare collected
2. Fetch data from the view

Task 11: Real-Life Scenario (Logic Building)

Senior citizens (age ≥ 60) get **20% discount** on ticket fare.

- Write a query to identify eligible passengers
- Write an UPDATE query to apply the discount
- Explain your logic in 2–3 lines

Submission Guidelines

- Submit **.sql file**
- Use comments for each task
- Queries must execute without errors

Bonus Tasks (Optional)

1. Add an **index on phone number**
2. Add a **CHECK constraint** to prevent same source and destination
3. Explain difference between **INNER JOIN** and **LEFT JOIN** using this database